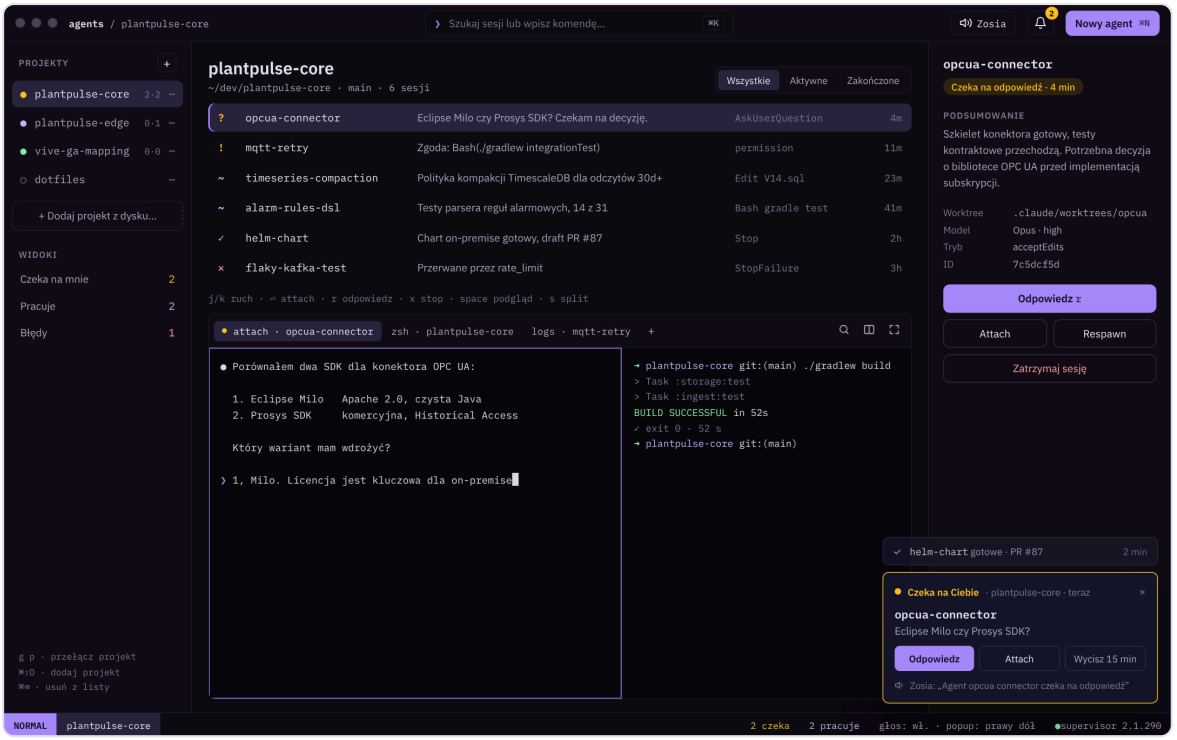
DOKUMENTACJA KONCEPCYJNA · WERSJA 0.1

Agent Manager

Natywne narzędzie na macOS do zarządzania lokalnymi agentami Claude Code: projekty, sesje, stany, terminal i powiadomienia w jednym oknie.

18.10.2026	**Krystian**	**Koncepcja zatwierdzona**	**macOS 15 Sequoia+**
data	autor i właściciel produktu	status	platforma



Wybrany ekran główny: Mission Control w stylu Keyboard-first, schemat Plex Violet, typografia IBM Plex.

Kluczowe decyzje

OBSZAR	DECYZJA
Stack	Kotlin + Compose Multiplatform for Desktop; terminal JediTerm + pty4j; hooki przez Ktor
Backend agentów	Oficjalny supervisor sesji w tle Claude Code (Agent View, v2.1.139+), sterowany przez CLI. Aplikacja nie trzyma procesów agentów
Zdarzenia	Hooki HTTP Claude Code → lokalny serwer na `127.0.0.1`; polling `claude agents --json` jako źródło prawdy
Ekran główny	Mission Control (3 panele) z paletą ⌘K, glifami stanu ? ! ~ ✓ × i skrótami j/k
Powiadomienia	Własny popup (pozycja, czas, akcje) + głos macOS (`say`, Zosia), konfiguracja per zdarzenie i projekt
Wygląd	9 schematów kolorów × 8 typografii, wybierane niezależnie, z podglądem na żywo
Architektura kodu	Modularny monolit: feature jako `api` / `impl` / `ui`, porty i adaptery, jeden composition root; testy jednostkowe, kontraktowe, funkcjonalne i architektury

1. Problem, cel i zakres

Przy kilku agentach Claude Code naraz praca rozjeżdża się po oknach terminala. Trudno zauważyć, który agent czeka na odpowiedź lub zgodę, a który skończył albo się wysypał. Sesje są rozproszone po projektach i worktree.

Co robi narzędzie

1. Trzyma listę **projektów** (lokalnych katalogów), które użytkownik dodaje i usuwa. Usunięcie nie kasuje plików.
2. Pokazuje **sesje agentów** ze stanem i krótkim podsumowaniem.
3. Pozwala **startować, zatrzymywać, wznawiać** agentów i **odpowiadać** im bez szukania okna.
4. Ma **pełnoprawny terminal**: attach do agenta, shell projektu, logi, panele, szukanie.
5. **Powiadamia** popupem i głosem, gdy agent czegoś potrzebuje.
6. Jest **szybkie z klawiatury** i **konfigurowalne wizualnie**.

Cele wersji 1.0

- Informacja, że agent czeka, w ≤ 2 s (popup lub głos).
- Odpowiedź i zgoda z popupu lub panelu, bez przełączania okna.
- Nowy agent w ≤ 3 akcjach (⌘N → prompt → Enter).
- Agenci pracują dalej po zamknięciu aplikacji.
- Cała obsługa możliwa z klawiatury.

Poza zakresem 1.0

- Własna pętla agenta lub klient API.
- Sesje w chmurze i dostęp z telefonu.
- Edytor kodu i przegląd diffów (to robi IntelliJ IDEA).
- Windows i Linux.
- Mac App Store (aplikacja działa bez App Sandbox).

Wymagania w skrócie

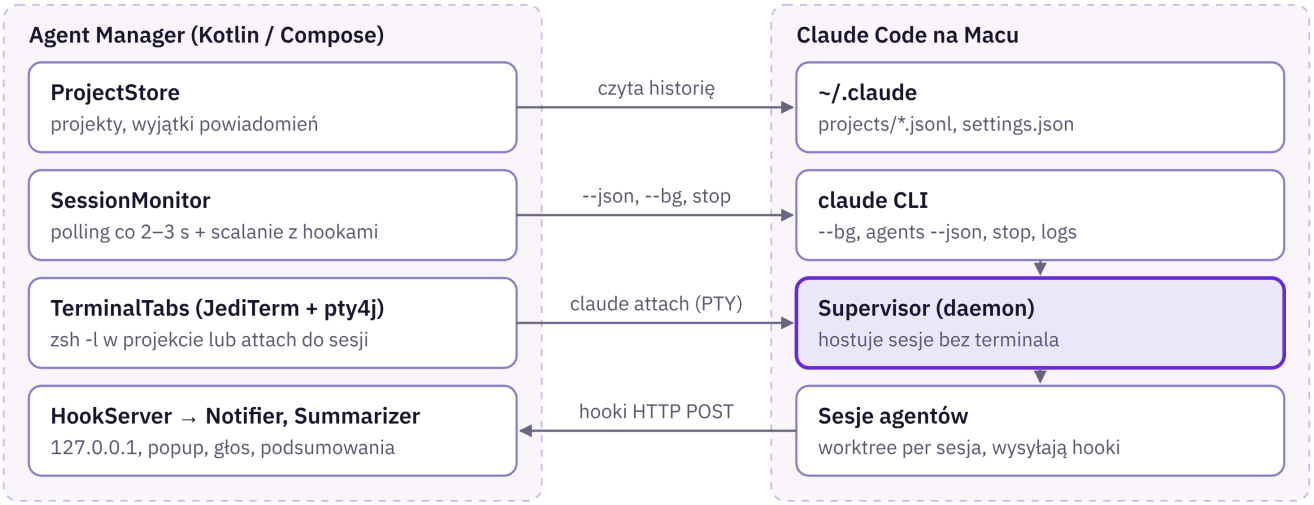
OBSZAR	MVP (MUST)	WERSJA 1.0 I PÓŹNIEJ
Projekty	Dodaj katalog (⌘⇧O), usuń z narzędzia (⌘⌫), wybór losu działających sesji przy usuwaniu	Walidacja katalogu (git, CLAUDE.md, hooki), import z `~/.claude/projects`
Sesje	Lista ze stanem i podsumowaniem; start (`claude --bg`), stop, respawn, usuń	Odpowiedź i zgoda bez terminala, filtry, streszczenia Haiku
Terminal	PTY: `zsh -l` i `claude attach`	Panele, szukanie regex, bloki komend, linki do IDE, broadcast
Powiadomienia	Macierz zdarzeń × kanałów, popup, głos z szablonami	Grupowanie, ciche godziny, tryb Skupienia, wyjątki per projekt
Wygląd i obsługa	9 schematów, 8 typografii, paleta ⌘K, skróty j/k	Podgląd na żywo, dzień/noc, własne motywy i `theme.json`

Najważniejsze wymagania niefunkcjonalne: stan odświeżany co ≤ 3 s; serwer hooków tylko na `127.0.0.1` z tokenem; edycja `~/.claude/settings.json` tylko za zgodą i z backupem; kontrast ≥ 4.5:1 w każdym schemacie; integracja z CLI za jednym interfejsem, bo Agent View jest w research preview.

2. Architektura

Aplikacja jest cienką nakładką GUI na supervisor sesji Claude Code. Steruje nim przez CLI, a o zdarzeniach dowiaduje się z hooków HTTP. Polling `claude agents --json` jest źródłem prawdy, a hooki dają natychmiastowe zdarzenia i treść (pytanie agenta, podsumowanie tury).

Aplikacja steruje CLI; agenci żyją w supervisorze Claude Code



Integracja z Claude Code

FUNKCJA	KOMENDA
Start agenta	`claude --bg --name "…" [--agent …] [--model …] "<prompt>"`
Lista sesji ze stanem	`claude agents --json --all [--cwd <projekt>]`
Terminal sesji / logi	`claude attach <id>` · `claude logs <id>`
Stop / restart / usunięcie	`claude stop` · `claude respawn` · `claude rm`
Hooki HTTP	`Notification`, `Stop` (`last_assistant_message`), `StopFailure`, `SessionStart`, `SessionEnd`

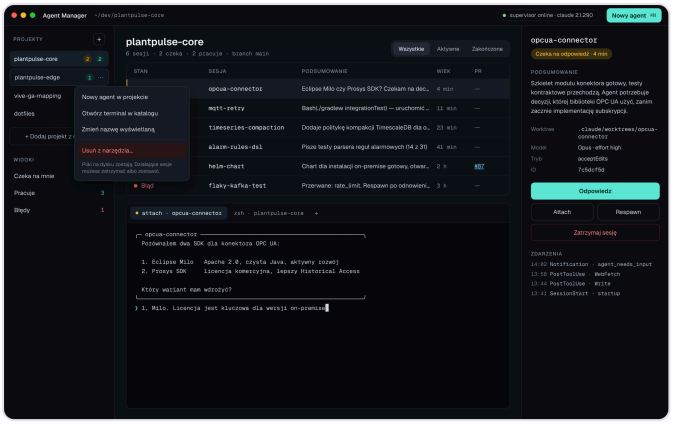
Stany sesji

GLIF	STAN W UI	Z `--JSON`	HOOK
?	Czeka na odpowiedź	`blocked` / `input needed`	`agent_needs_input`
!	Prosi o zgodę	`blocked` / `permission prompt`	`permission_prompt`
~	Pracuje	`working`	`PreToolUse`
✓	Gotowe	`done`	`Stop`
×	Błąd	`failed`	`StopFailure`

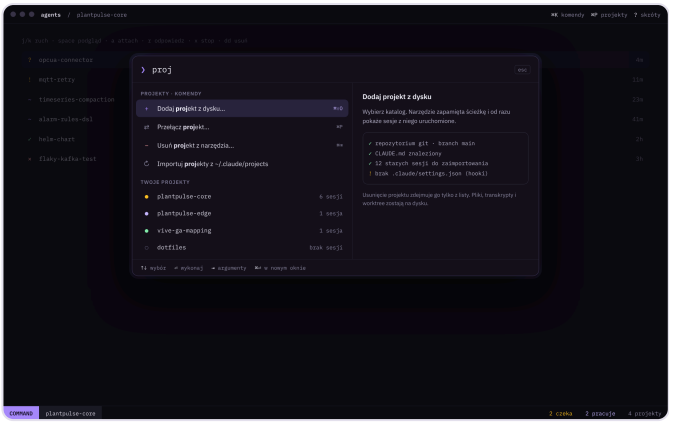
Moduły: `ClaudeCli`, `ProjectStore`, `SessionMonitor`, `HookServer`, `Summarizer`, `Notifier`, `TerminalTabs`, `ThemeManager`, `CommandPalette`. Konfiguracja w `~/Library/Application Support/AgentManager/*.json`.

3. Koncepcje UX i wybór

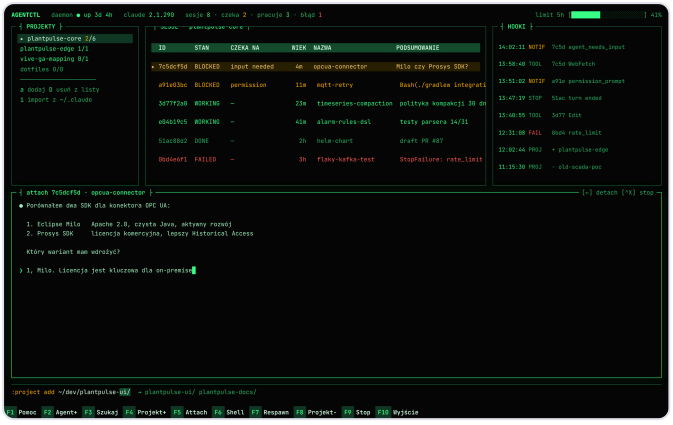
Powstało 5 kierunków z tymi samymi funkcjami, ale innym sposobem pracy.



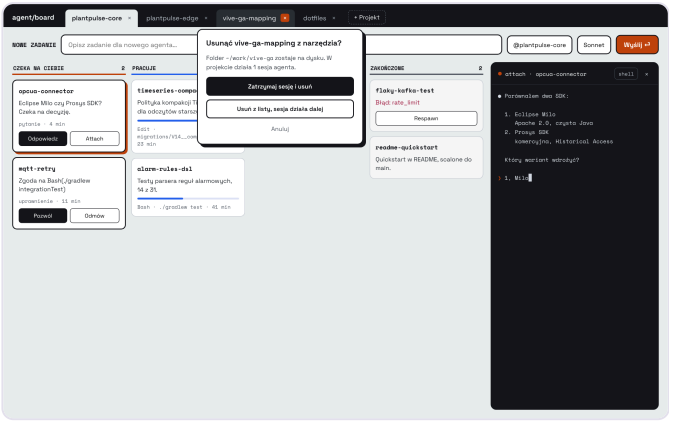
1 · Mission Control — trzy panele, terminal pod listą, szczegóły sesji



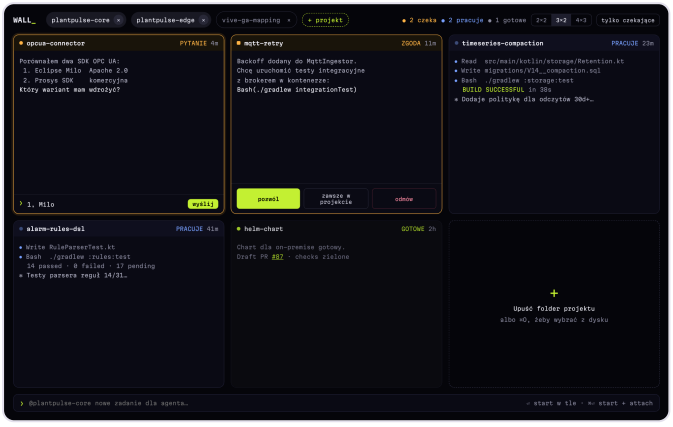
2 · Keyboard-first — paleta ⌘K, skróty vima, pasek trybu



3 · TUI / htop — fosfor, ramki, log hooków, klawisze F1–F10



4 · Kanban — kolumny stanów, projekty jako zakładki

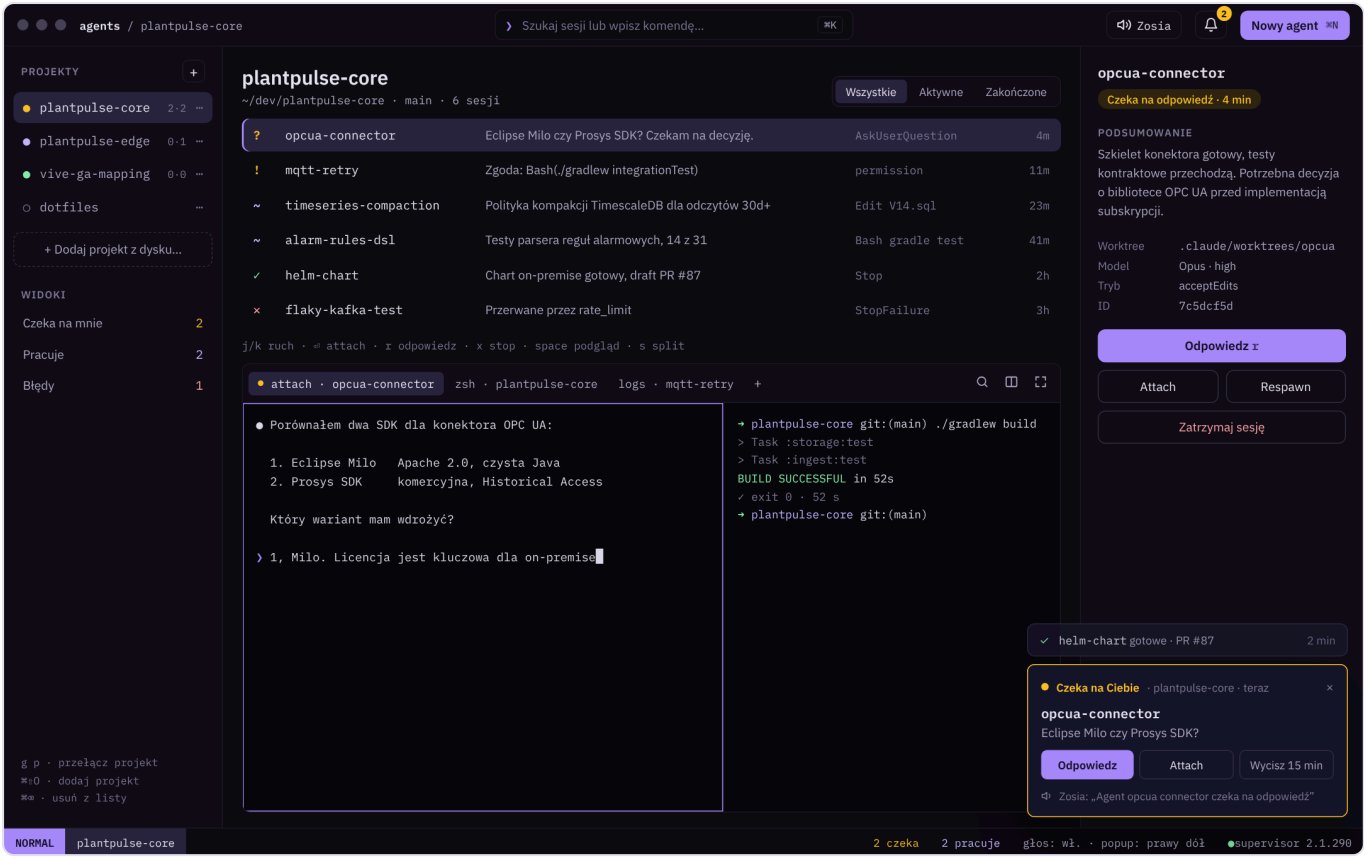


5 · Ściana terminali — siatka żywych kafelków z odpowiedzią w kafelku

Wybór: Mission Control w stylu Keyboard-first.

- Z Mission Control: układ trzech paneli i panel szczegółów.
- Z Keyboard-first: paleta ⌘K, glify stanu, skróty j/k, pasek trybu, IBM Plex i fiolet.
- Ze ściany terminali: odpowiedź bez otwierania terminala (popup, panel).
- Z Kanbana: dialog usuwania projektu z działającymi sesjami.
- Z TUI: schemat Phosphor i JetBrains Mono jako opcje wyglądu.

4. Mission Control — ekran główny



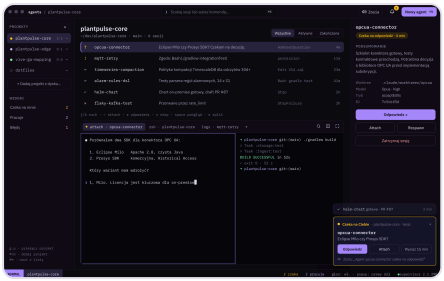
Nagłówek z paletą ⌘K, sidebar projektów, lista sesji, terminal z dwoma panelami, panel szczegółów, pasek trybu i popup „czeka na Ciebie" z podpisem komunikatu głosowego.

STREFA	ZAWARTOŚĆ
Nagłówek	Paleta ⌘K, głos, dzwonek, Nowy agent ⌘N
Sidebar	Projekty z licznikami, ⋯, dodaj z dysku, widoki
Środek	Lista sesji, filtr, terminal z kartami
Szczegóły	Podsumowanie, worktree, model, Odpowiedz, Attach, Respawn, Stop
Pasek trybu	NORMAL, projekt, liczniki, głos i popup, supervisor

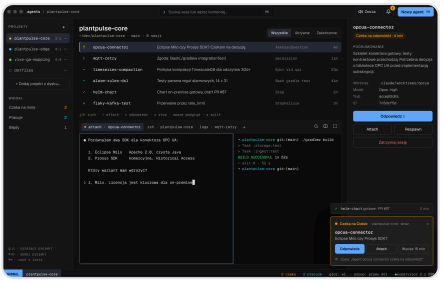
SKRÓT	AKCJA
⌘K · ⌘N	Paleta komend · nowy agent
⌘⇧O · ⌘⌫	Dodaj · usuń projekt
j / k · g p	Ruch po sesjach · przełącz projekt
⏎ · r · x	Attach · odpowiedz · stop
space · s	Podgląd · podziel terminal

5. Wygląd: 9 schematów × 8 typografii

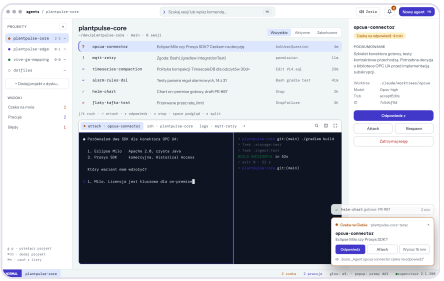
Schemat kolorów i typografię wybiera się niezależnie, w *Ustawienia → Wygląd*, z podglądem prawdziwego ekranu na żywo. Poniżej wszystkie schematy, każdy z przykładową typografią.



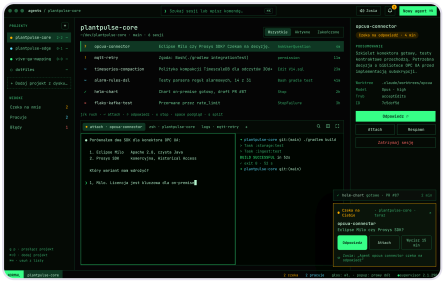
Plex Violet · IBM Plex



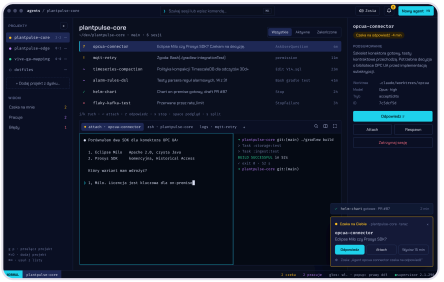
Graphite · Geist



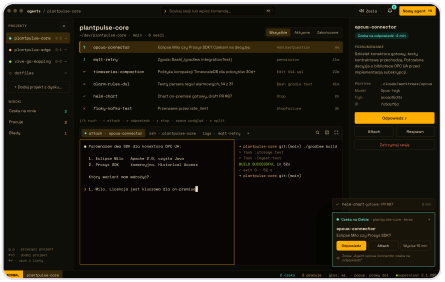
Paper Indigo · Instrument + DM Mono



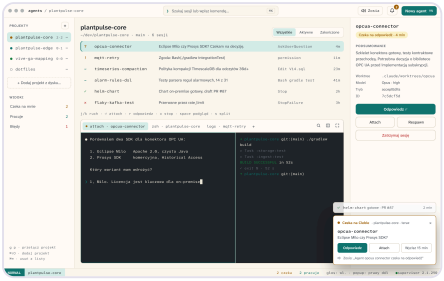
Phosphor · JetBrains Mono



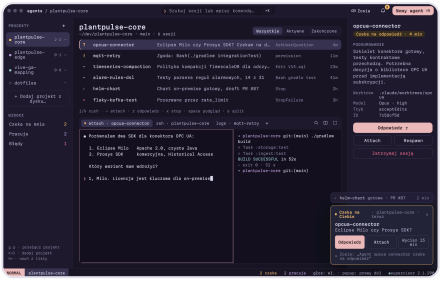
Synth · Chakra Petch + Fira Code



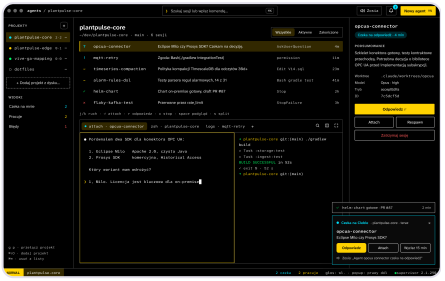
Amber CRT · Space Grotesk + Space Mono



Sand · Atkinson Hyperlegible



Rose Night · Martian Mono



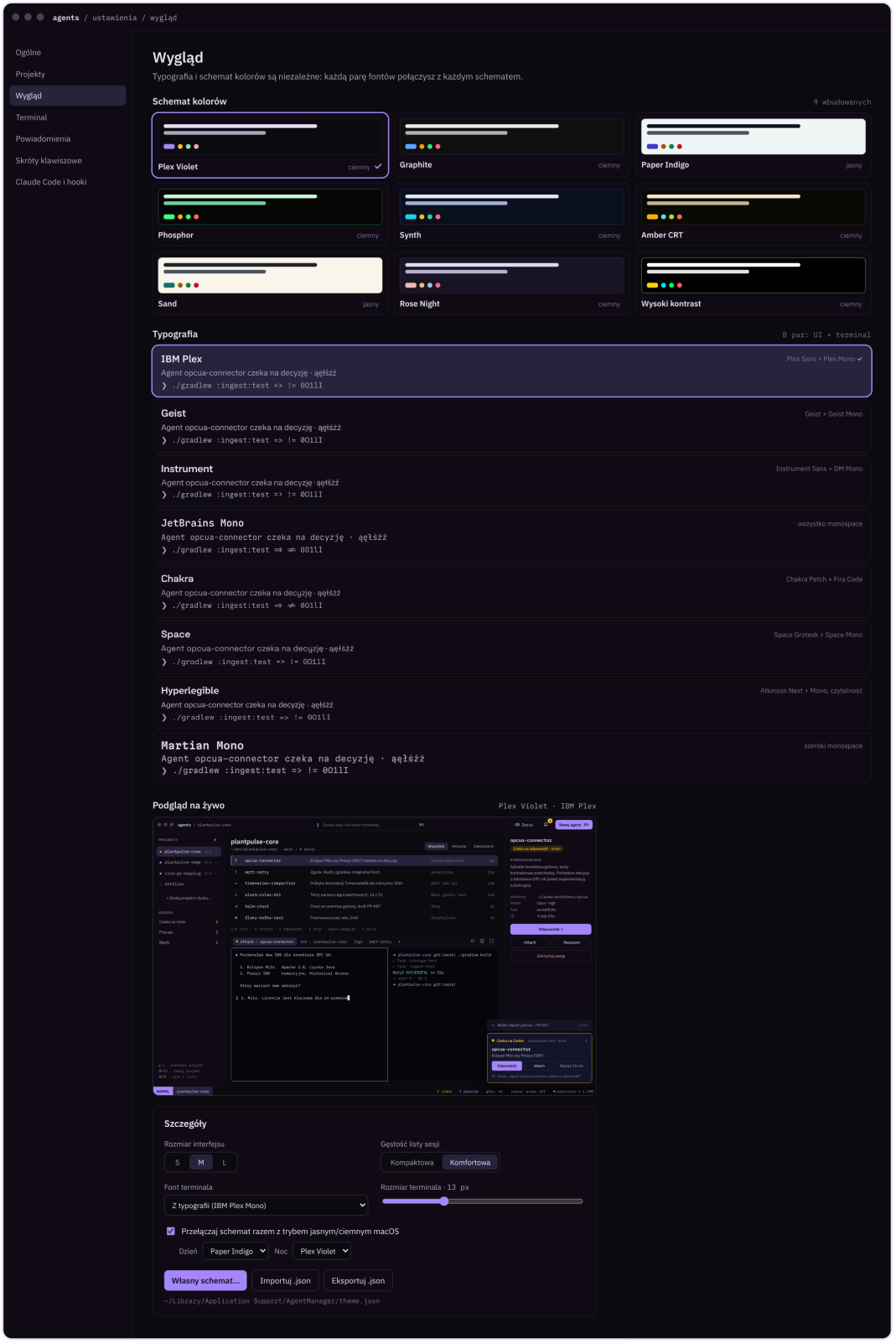
Wysoki kontrast · Atkinson Hyperlegible

Typografie (font UI + font terminala)

NAZWA	FONTY	CHARAKTER
IBM Plex (domyślna)	Plex Sans + Plex Mono	techniczna, neutralna
Geist	Geist + Geist Mono	nowoczesna, zwarta
Instrument	Instrument Sans + DM Mono	miękka, „produktowa”
JetBrains Mono	JetBrains Mono	wszystko monospace, ligatury
Chakra	Chakra Petch + Fira Code	kanciasta, „sci-fi”
Space	Space Grotesk + Space Mono	ekspresyjna, retro
Hyperlegible	Atkinson Hyperlegible Next + Mono	maksymalna czytelność
Martian Mono	Martian Mono	szeroki monospace

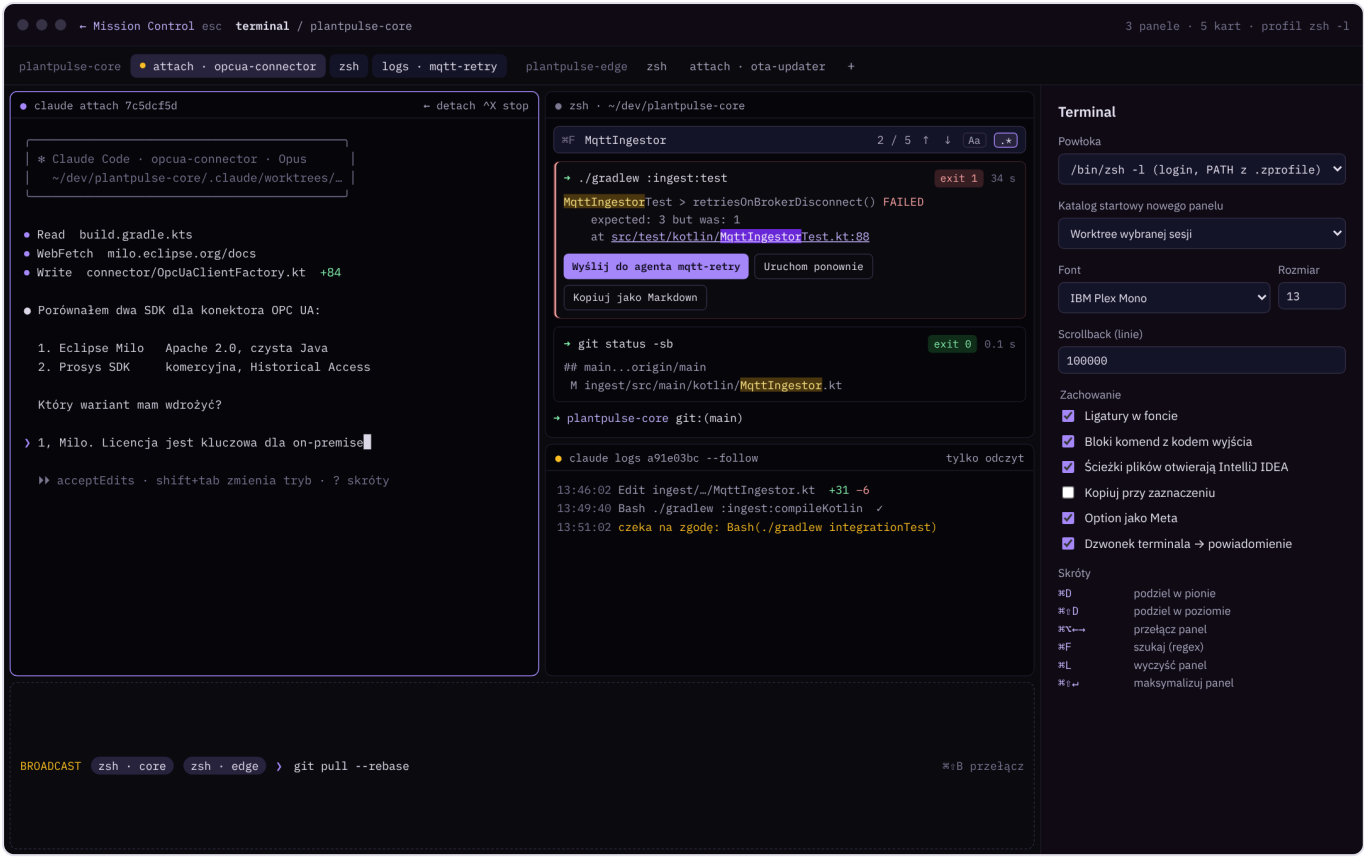
Wszystkie fonty na licencji SIL OFL 1.1, mogą być dołączone do aplikacji. Dodatkowo: rozmiar interfejsu S/M/L, gęstość listy, font i rozmiar terminala, przełączanie dzień/noc z macOS, własne motywy w `theme.json`.

6. Ustawienia wyglądu



Wybór schematu i typografii, podgląd Mission Control na żywo i szczegółowe ustawienia.

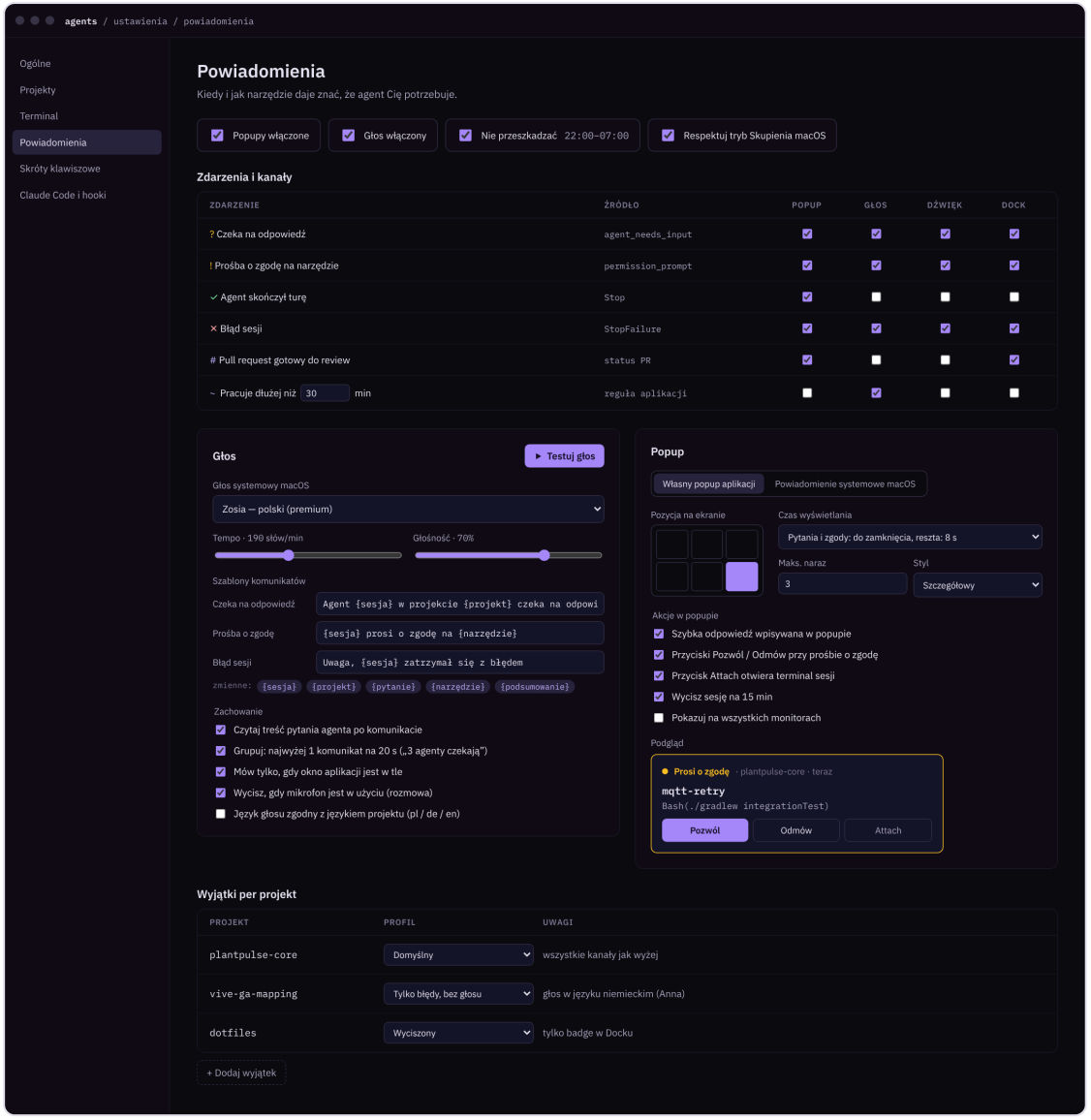
7. Terminal



Tryb terminala: karty per projekt, trzy panele (attach agenta, zsh z blokami komend i szukaniem, logi), broadcast i panel ustawień.

- **Karty per projekt:** `attach · sesja`, `zsh`, `logs · sesja`.
- **Panele:** ⌘D, ⌘⇧D, przełączanie ⌘⌥←→, maksymalizacja ⌘⇧↵.
- **Szukanie** ⌘F z licznikiem, wielkością liter i regex.
- **Bloki komend** z kodem wyjścia i czasem; akcja „Wyślij do agenta” przekazuje błąd testu do sesji.
- **Linki** `plik.kt:88` otwierają IntelliJ IDEA.
- **Broadcast** jednej komendy do kilku paneli (⌘⇧B).
- **Ustawienia:** powłoka (`zsh -l`), katalog startowy, font, scrollback 100 000, ligatury, Option jako Meta, dzwonek → powiadomienie.
- **Implementacja:** JediTerm + pty4j w `SwingPanel`; bloki przez znaczniki OSC 133.

8. Powiadomienia: popup i głos



Macierz zdarzeń i kanałów, ustawienia głosu i popupu, wyjątki per projekt.

8. Powiadomienia — ustawienia domyślne

ZDARZENIE	ŹRÓDŁO	POPUP	GŁOS	DŹWIĘK	DOCK
Czeka na odpowiedź	`agent_needs_input`	✓	✓	✓	✓
Prośba o zgodę	`permission_prompt`	✓	✓	✓	✓
Tura zakończona	`Stop`	✓	–	–	–
Błąd sesji	`StopFailure`	✓	✓	✓	✓
PR gotowy do review	status PR	✓	–	–	✓
Pracuje dłużej niż 30 min	reguła aplikacji	–	✓	–	–

Popup

- Własny (okno Compose, zawsze na wierzchu) albo systemowy macOS.
- Pozycja: 6 miejsc, domyślnie prawy dół.
- Pytania i zgody do zamknięcia, reszta 8 s; maks. 3 naraz.
- Akcje: szybka odpowiedź, Pozwól/Odmów, Attach, Wycisz 15 min.

Głos

- `say -v Zosia -r 190`; także Anna (de), Samantha (en).
- Szablony ze zmiennymi `{sesja}`, `{projekt}`, `{pytanie}`, `{narzędzie}`.
- Najwyżej 1 komunikat na 20 s, kolejne łączone.
- Cisza przy aktywnym mikrofonie, w godzinach nocnych i w trybie Skupienia.

Wyjątki per projekt: np. projekt służbowy tylko z błędami i głosem po niemiecku, dotfiles wyciszone (tylko badge w Docku).

9. Plan MVP, ryzyka i otwarte kwestie

#	ETAP	WYNIK
0	Spike: Compose + JediTerm, `claude agents --json`	Okno z terminalem i surową listą sesji
1	Szkielet i `ClaudeCli`, `SessionMonitor`	Lista sesji odświeżana co 2–3 s
2	Projekty: dodaj, waliduj, usuń	Sidebar projektów
3	Mission Control, paleta ⌘K, skróty	Start/stop/respawn z klawiatury
4	Terminal: karty, panele, szukanie	Attach i shell w aplikacji
5	Hooki i podsumowania	Stan w ≤ 1 s, podsumowania tur
6	Powiadomienia: popup, głos, reguły	Kompletne MVP
7	Wygląd: schematy, typografie, podgląd	Ustawienia wyglądu
8	Szlif, Dock, menu bar, notaryzacja, `.dmg`	Wersja 1.0

Główne ryzyka

- Agent View w research preview: format CLI może się zmienić → wszystko za `ClaudeCli`, testy kontraktowe.
- Brak oficjalnej komendy `reply` → odpowiedź przez attach; zbadać Agent SDK.
- JediTerm w Compose (fokus, skróty, HiDPI) → spike w etapie 0.
- Token hooków musi dotrzeć do procesów supervisora → sprawdzić dziedziczenie env.

Otwarte decyzje

- Hooki globalnie czy per projekt?
- Odpowiedź przez attach czy własny widok czatu (Agent SDK)?
- Streszczenia Haiku: domyślnie włączone?
- Popup systemowy macOS w 1.0?
- Nazwa produktu i ikona.

Kryterium MVP: dodaję projekt, startuję agenta, a gdy zada pytanie, w ≤ 2 s słyszę komunikat i widzę popup, odpowiadam z popupu, zamykam aplikację, a agent pracuje dalej.

10. Wymagania inżynierskie

Cel nadrzędny: zmiana w jednym miejscu nie wymusza zmian w innych. Zasady są wiążące dla każdego kodu, także pisanego przez agentów Claude Code.

OBSZAR	WYMAGANIA (SKRÓT)
Modułowość i hermetyczność	Modularny monolit; każdy feature jako `api` / `impl` / `ui`. `impl` widzi tylko `app`; wszystkie klasy w `impl` są `internal` poza fabryką modułu. `api` zależy tylko od `core:domain`. Brak cykli, brak globalnego stanu. Kod systemowy (procesy, pliki, HTTP, `say`) wyłącznie w adapterach.
Interfejsy	KDoc na każdym publicznym typie; małe interfejsy (odczyt osobno od komend); typy niemutowalne, ID jako `value class`; błędy jako typowane `Outcome`, nie wyjątki; IO tylko przez `suspend` i `Flow`; typy bibliotek nie wyciekają do `api`; fake'i w `testFixtures`.
Clean code	Kotlin Coding Conventions, ktlint + detekt bez ostrzeżeń; funkcja ≤ 30 linii, złożoność ≤ 10, plik ≤ 400 linii; zakaz `!!`, `GlobalScope`, `runBlocking`; funkcyjny rdzeń, imperatywna powłoka; wstrzykiwane dispatchery i `Clock`; ADR dla decyzji; Conventional Commits, PR ≤ 400 linii.
Testy	Jednostkowe (≥ 80% linii per `impl`, ≥ 90% rdzeni), kontraktowe dla 100% interfejsów `api`, integracyjne dla każdego adaptera, funkcjonalne dla każdego kryterium akceptacji, testy architektury w każdym buildzie. Hermetyczne i deterministyczne: bez sieci, bez prawdziwego Claude Code, wirtualny czas.
Bramka i DoD	`./gradlew check` zielony; nowa logika z testami; KDoc i ADR; PR wymienia zmienione moduły i uzasadnia każdy spoza feature; dokumentacja aktualna.

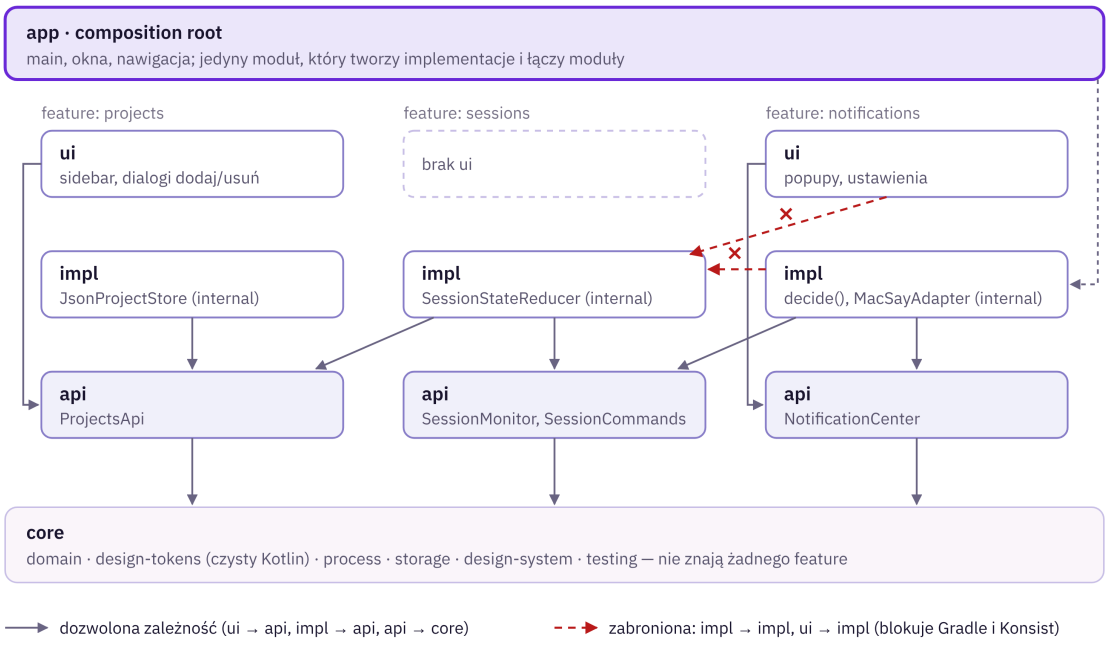
Testy w skrócie

POZIOM	CO SPRAWDZA	PRZYKŁAD
Architektura	Reguły zależności, widoczność	Konsist: `impl` nie importuje cudzego `impl`
Jednostkowe	Czyste rdzenie, tablicowo	Reducer stanu sesji, `decide()` powiadomień, kontrast schematów ≥ 4.5:1
Kontraktowe	Fake = prawdziwa implementacja	`ProjectsApiContract` na fake i na `JsonProjectStore`
Integracyjne	Adaptery na lokalnych zasobach	Fałszywy plik `claude` na `PATH`, serwer hooków na losowym porcie
Funkcjonalne	Scenariusze przez UI całej aplikacji	„Agent pyta → popup i głos w ≤ 2 s → odpowiedź z popupu”
E2E	Prawdziwy Claude Code	Nocą, tag `e2e`, poza bramką PR

11. Koncepcja techniczna

Modularny monolit w stylu portów i adapterów. Feature rozmawiają tylko przez interfejsy `api`; implementacje składa jeden composition root w module `app`. Reguły wymusza Gradle (convention plugins) i Konsist.

Feature zależą od siebie tylko przez api; implementacje widzi wyłącznie app



Pozostałe feature (agent-runtime, summaries, terminal, appearance, command-palette, mission-control) mają ten sam układ.

Feature

FEATURE	KONTRAKT (`API`)
agent-runtime	`AgentRuntime`, `AgentEventSource`, `HookInstaller`
projects	`ProjectsApi`
sessions	`SessionMonitor`, `SessionCommands`
summaries	`Summarizer`
notifications	`NotificationCenter`, ustawienia
terminal	`TerminalApi`, `TerminalSession`
appearance	`AppearanceApi`, katalog
command-palette	`CommandRegistry`

Izolacja zmian

ZMIANA	DOTYKA TYLKO
Nowy format `agents --json`	`agent-runtime:impl`
CLI → Agent SDK	nowe `AgentRuntime` + `app`
`say` → AVSpeech	adapter w `notifications:impl`
Inna biblioteka terminala	`terminal:impl`
Nowy schemat kolorów	katalog w `appearance:impl`
Nowy format `projects.json`	migracja w `projects:impl`
Przeprojektowany ekran	`mission-control:ui`

Stack: Gradle Kotlin DSL + `build-logic`, JDK 21, Kotlin 2.x, Compose Desktop, coroutines/Flow, kotlinx.serialization, Ktor, JediTerm + pty4j, Spotless/ktlint, detekt, Kover, Konsist, JUnit 5, Kotest, Turbine. Ręczne DI, błędy jako `Outcome`, konfiguracja w plikach JSON z jednym właścicielem i migracjami schematu.

Pełna dokumentacja: `README.md` i pliki `01–12` w paczce `agent-manager-docs`. Źródła: dokumentacja Claude Code (Agent View, Hooks reference, Headless mode), code.claude.com i docs.claude.com.